



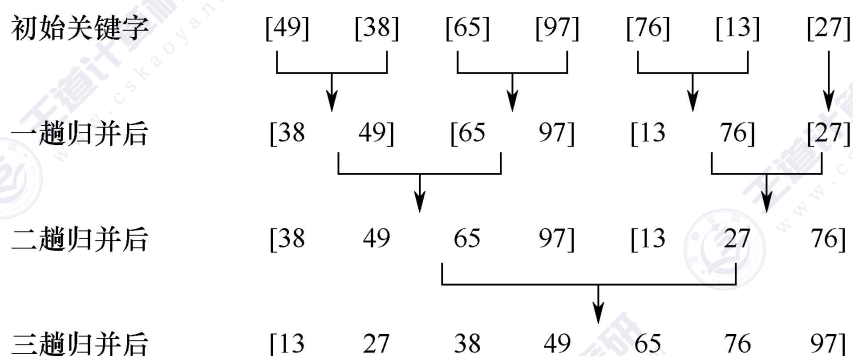
本小节内容

归并排序原理解析

归并排序代码实战

所有排序算法时间与空间复杂度汇总

归并排序原理解析



如上图所示，我们把每两个元素归为一组，进行小组内排序，然后再次把两个有序小组合并为一个有序小组，不断进行，最终合并为一个有序数组。

接下来我们借助动画网站来理解一下

<https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

动画使用方法是，先点 play，然后及时点击 pause，自己通过 Step Forward 来查看

归并排序代码实战

归并排序的代码是采用递归思想实现的，考研掌握递归实现即可。首先，最小下标值和最大下标值相加并除以 2，得到中间下标值 mid，用 MergeSort 对 low 到 mid 排序，然后用 MergeSort 对 mid+1 到 high 排序。当数组的前半部分和后半部分都排好序后，使用 Merge 函数。Merge 函数的作用是合并两个有序数组。为了提高合并有序数组的效率，在 Merge 函数内定义了 B[N]。首先，我们通过循环把数组 A 中从 low 到 high 的元素全部复制到 B 中，这时游标 i（遍历的变量称为游标）从 low 开始，游标 j 从 mid+1 开始，谁小就将谁先放入数组 A，对其游标加 1，并在每轮循环时对数组 A 的计数游标 k 加 1。

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define N 7
```

```
typedef int ElemType;
```

```
//49,38,65,97,76,13,27
```



```
void Merge(ElemType A[],int low,int mid,int high)
{
    static ElemType B[N]; //加 static 的目的是无论递归调用多少次，都只有一个 B[N]
    int i,j,k;
    for(k=low;k<=high;k++) //复制元素到 B 中
        B[k]=A[k];
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++) //合并两个有序数组
    {
        if(B[i]<=B[j])
            A[k]=B[i++];
        else
            A[k]=B[j++];
    }
    while(i<=mid) //如果有剩余元素，接着放入即可
        A[k++]=B[i++]; //前一半的有剩余的放入
    while(j<=high)
        A[k++]=B[j++]; //后一半的有剩余的放入
    }
    //归并排序不限制是两两归并，还是多个归并，考研都是考两两归并
void MergeSort(ElemType A[],int low,int high) //递归分割
{
    if(low<high)
    {
        int mid=(low+high)/2;
        MergeSort(A,low,mid); //排序好前一半
        MergeSort(A,mid+1,high); //排序好后一半
        Merge(A,low,mid,high); //讲两个排序好的数组合并
    }
}

void print(int* a)
{
    for(int i=0;i<N;i++)
    {
        printf("%3d",a[i]);
    }
    printf("\n");
}

//《王道 C 督学营》课程
// 归并排序
int main()
{
    int A[7]={49,38,65,97,76,13,27}; //数组，7 个元素
    MergeSort(A,0,6);
    print(A);
}
```



```
return 0;
}
```

MergeSort 函数的递归次数是 $\log_2 n$ ，Merge 函数的循环了 n 次，因此时间复杂度是 $O(n \log_2 n)$ 。
归并排序最好、最坏、平均时间复杂度都是 $O(n \log_2 n)$ 。

归并排序的空间复杂度是 $O(n)$ ，因为使用了数组 B，它的大小与 A 一样，占用 n 个元素的空间。

所有排序算法时间与空间复杂度汇总

各种排序算法的时间复杂度、空间复杂度、稳定性和复杂性

排序方式	时间复杂度			空间复杂度	稳 定 性	复 杂 性
	平均情况	最坏情况	最好情况			
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	简单
希尔排序	$O(n^{1.3})$			$O(1)$	不稳定	较复杂
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	简单
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(\log_2 n)$	不稳定	较复杂
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	简单
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定	较复杂
归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	稳定	较复杂
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定	较复杂

稳定性是指排序前后，相等的元素位置是否会被交换

复杂性是指代码编写的难度